

Doc. no.:	P0849R2
Date:	2019-10-06
Audience:	EWG, LEWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

auto(x): *decay-copy* in the language

Changes Since R1

- propose `decltype(auto)(x)` as well

Changes Since R0

- updated examples
- discussed `decltype(auto)(x)`
- added library wording

Introduction

This paper proposes `auto(x)` and `auto{x}` for casting `x` into a prvalue value as if passing `x` as a function argument by value. The functionality is realized as the *decay-copy* function in the standard for exposition only.

The paper also proposes `decltype(auto)(x)` and `decltype(auto){x}` for forwarding `x`.

Motivation

Obtaining a prvalue copy is necessary

A generic way to obtain a copy of an object in C++ is `auto a = x;` but such a copy is an lvalue. We could often convey the purpose in code more accurately if we can obtain a copy as a prvalue. In the following example, let `Container` be a concept,

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), auto(x.front()));
}
```

If we wrote

```
void pop_front_alike(Container auto& x) {
    auto a = x.front();
    std::erase(x.begin(), x.end(), a);
}
```

, questions arise – why this is not equivalent to

```
void pop_front_alike(Container auto& x) {
    std::erase(x.begin(), x.end(), x.front());
}
```

The problem is, the statement to obtain an lvalue copy is a declaration:

```
auto a = x.front();
```

Its primary purpose is to declare a variable while being a copy is a property of the declaration. In contrast, the expression to obtain an rvalue copy is a clear command to perform a copy:

```
auto(x.front())
```

One might argue that the above is indifferent from

```
T(x.front())
```

However, there are plenty of situations that the `T` is nontrivial to get. We probably don't want to write the original example as

```
void pop_front_alike(Container auto& x) {
    using T = std::decay_t<decltype(x.front())>;
    std::erase(x.begin(), x.end(), T(x.front()));
}
```

Obtaining a prvalue copy with `auto(x)` works always

In standard library specification, we use the following exposition only function to fulfill the role of `auto(x)` :

```
template<class T>
constexpr decay_t<T> decay_copy(T&& v) noexcept(
    is_nothrow_convertible_v<T, decay_t<T>>) {
    return std::forward<T>(v);
}
```

This definition involves templates, dependent `constexpr` , forwarding reference, `noexcept` , and two traits, and still has caveats if people want to use it in practice. An obvious issue is that `decay_copy(x.front())` creates a copy of `x.front()` even if `x.front()` is already a prvalue (thus, already a copy).

There is a less obvious issue which needs a minimal reproduce:

```
class A {
    int x;

public:
    A();

    auto run() {
        f(A(*this));           // ok
        f(auto(*this));        // ok as proposed
        f(decay_copy(*this));  // ill-formed
    }

protected:
    A(const A&);
};
```

The problem is that `decay_copy` is nobody's friend. We can use `A` directly in this specific example, but in a more general setting, where a type `A` has access to a set of type `T`'s private or protected copy/move constructors, *decay-copy* an object of `T` fails inside `A`'s class scope, but `auto(x)` continues to work.

Constrained auto wants to forward without T

On CppCon 2019, the author observed that multiple speakers are using `std::forward<decltype(arg)>(arg)` in their slides. The context is that in generic functions that use the constrained `auto` syntax, there is no access to the type τ that “declared” `arg` :

```
auto f(Copyable auto&& arg) {
    /* ... */ std::forward<??>(arg);
}
```

`decltype(auto){arg}` can forward `arg` without computing `arg` 's type. It is equivalent to `static_cast<decltype(arg)>(arg)` . If `arg` is a variable of type `T&&` , `arg` is an lvalue but `static_cast<T&&>(arg)` is an xvalue.

Discussion

auto(x) is a missing piece

Replacing the `char` in `char('a')` with `auto` , we obtain `auto('a')` , which is a function-style cast. Such a formula also supports *injected-class-names* and class template argument deduction in C++17. Introducing `auto(x)` and `auto{x}` significantly improves the language consistency:

variable definition	function-style cast	new expression
<code>auto v(x);</code>	<code>auto(x)</code>	<code>new auto(x)</code>
<code>auto v{x};</code>	<code>auto{x}</code>	<code>new auto{x}</code>
<code>ClassTemplate v(x);</code>	<code>ClassTemplate(x)</code>	<code>new ClassTemplate(x)</code>
<code>ClassTemplate v{x};</code>	<code>ClassTemplate{x}</code>	<code>new ClassTemplate{x}</code>

****** *The type of `x` is a specialization of `ClassTemplate` .*

With this proposal, all the cells in the table copy construct form `x` (given CTAD’s default behavior) to obtain lvalues, prvalues, and pointers to objects, categorized by their columns. Defining `auto(x)` as a library^[1] facility loses orthogonality.

Introducing `auto(x)` into the language even improves the library consistency:

type function style	expression style
<code>void_t<decltype(<i>expr</i>)></code>	<code>decltype(void(<i>expr</i>))</code>
<code>decay_t<decltype(<i>expr</i>)></code>	<code>decltype(auto(<i>expr</i>))</code>

decltype(auto){x} means the right thing

One problem in teaching `std::forward` is that we may have to explain why and how forwarding rvalues as lvalues are forbidden. Such a gap implies that the approach we took diverges from the meaning we want to express.

“Forwarding” means restoring an expression’s value category given its type. `decltype(auto){x}` does *precisely* that. `decltype(auto)` is the expression’s type, so `decltype(auto){x}` casts the expression into a different one whose value category matches the type.

`decltype(auto){x}` is not only a shorter way^[2] to forward arguments, but also means a right way to forward arguments.

Implementation

Try it out (not including `decltype(auto)(x)`): Godbolt

(<https://godbolt.miator.net/#z:OYLghAFBqd5QCxAYwPYBMCmBRdBLAF1QCcAaPECAKxAEZSAbAQwDtRkBSAJgCFufSAZ1QBXYSkwgA5NwDMeFsgYisAag6yAwi0wB3Ddg4AGAILGTBTAFsADs0vqtBAJ43MLJlcyqAKgfMAbqh46KoAZhB%2BXABS3NEAlBp8puYKBKpWTAoQiSkA7MkmqsWqyAhMxKpMHACsfDUALo5N3FwseBKtSeYlqmmlqCyCBHGqIRpNACxc3aa9EUwiRBBM8YmyhfMrS6gcBcR7DeuFPSWLywAe8c2qYFJMd7NFJSKCCsCq6ROqWEoubttlqtjuZQVx5GFVEZTsUFjsciC5iU4UQ9nw8kc nltzqglFfxSLI1ljkYDdgV6DMMYizEjijpdFudmjDklejDVPTGUDqb1OTi0ZSGsS6XoubiBYceSU%2BfD8YThRzRfyCvjJiyNmDZO58GFQRipPFGNialJSCxpEZTahpJp%2BPxVMlxBJ1ODaKaCBaDYaANYgGpGI1SSam81SS2ka1SU2CEABj1hg2kOCwJBoWx4BiYMgUCBpmwZrMgKzOJSsYCKMIZyzEGMQABGntldYUFWc0jdpDTXhYBAA8iwGG2E6QsJk2JnG/hiJhkaQ8AFMDHh5gLjOlplpB20pgGI2CMQ8FZPYbmGwUHbelw8HWY5BDagbHPBkuALQAdSYDAYqhfw3QE04Xh%2BC4Jho1EcRJFoE9jRDRtI2LUs2FUCBcEIEgXVkehVE0VB00zcZXWuW0gN4d1jx9P0A13INYOHSNo1jUh40tNZAy4Wjw3opjyNIBcazwQYQEmlA>)

Wording

The wording is relative to N4830.

Modify 7.6.1.3 [expr.type.conv]/1 as indicated:

A *simple-type-specifier* (9.1.8.2) or *typename-specifier* (13.7) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.3.1.8) for the remainder of this section. Otherwise, if the type is `auto` or `decltype(auto)`, it is replaced by the type deduced for the variable `x` in the invented declaration (9.1.8.5):

```
auto x init;
```

or

```
decltype(auto) x init;
```

, respectively, where *init* is the initializer.

Modify 9.1.8.5 [dcl.spec.auto]/5 as indicated:

A placeholder type can also be used in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (7.6.2.7) and as a *decl-specifier* of the *parameter-declaration's decl-specifier-seq* in a *template-parameter* (13.1). The `auto` and `decltype(auto)` *type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation) (7.6.1.3).

Remove the first entity from 16.4.2.1 [expos.only.func]/2:

```
template<class T> constexpr decay_t<T> decay_copy(T&& v)
    noexcept(is_nothrow_convertible_v<T, decay_t<T>>) // exposition only
    { return std::forward<T>(v); }
```

Search and replace “~~calls to `decay_copy` being evaluated in~~” (the thread/the current thread) with “where the values produced by `auto` are materialized in”.

Search and replace “~~`decay_copy`~~” with “`auto`”.

Acknowledgments

Thank Alisdair Meredith, Arthur O'Dwyer, and Billy O'Neal for providing examples and feedback for this paper.

References

1. Krügler, Daniel. P0758R0 *Implicit conversion traits and utility functions*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html>) ↩
2. Revzin, Barry. P0644R1 *Forward without forward* . <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0644r1.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0644r1.html>) ↩